

Fundamentals of Distributed Systems

DESIGNING RELIABLE APPLICATIONS



Michael L Perry

SOFTWARE MATHEMATICIAN

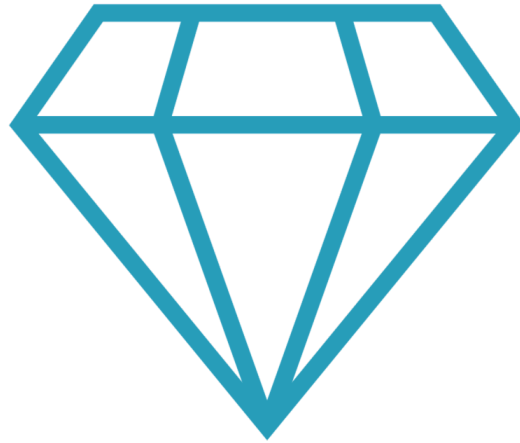
@michaelperry immutablearchitecture.com



Properties of a Reliable Application



Idempotence



Immutability



Location
Independence

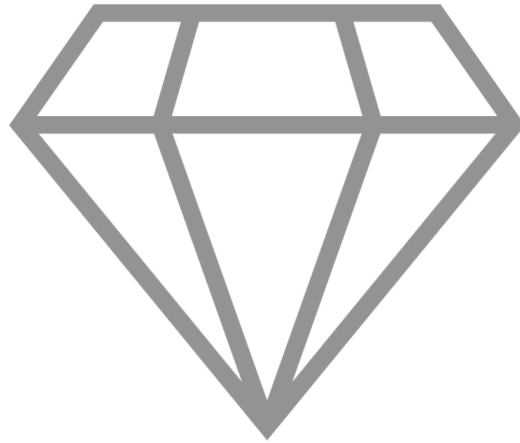


Versioning

Properties of a Reliable Application



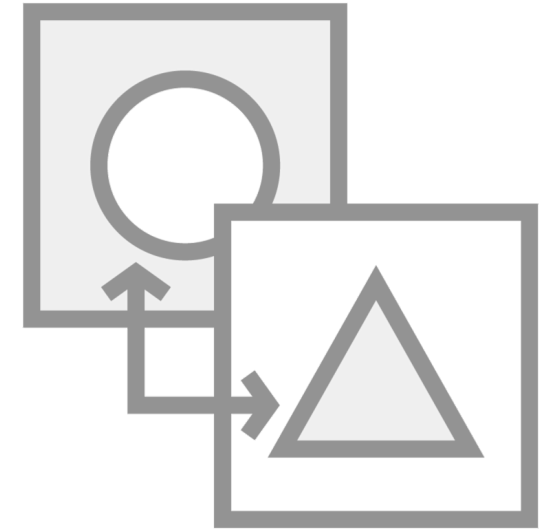
Idempotence



Immutability



**Location
Independence**

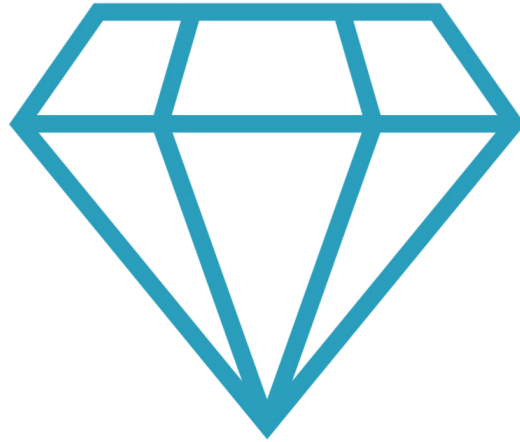


Versioning

Properties of a Reliable Application



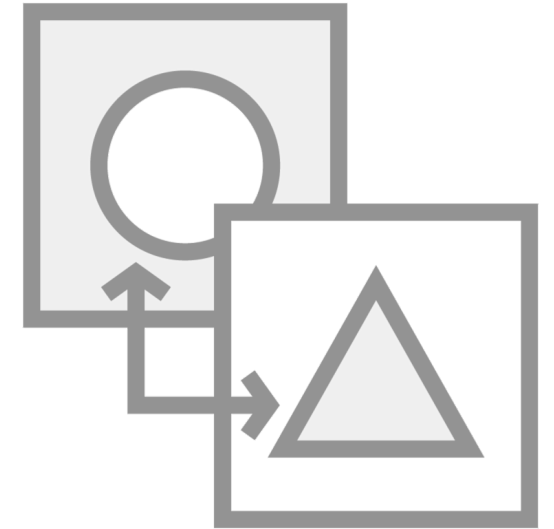
Idempotence



Immutability



Location
Independence

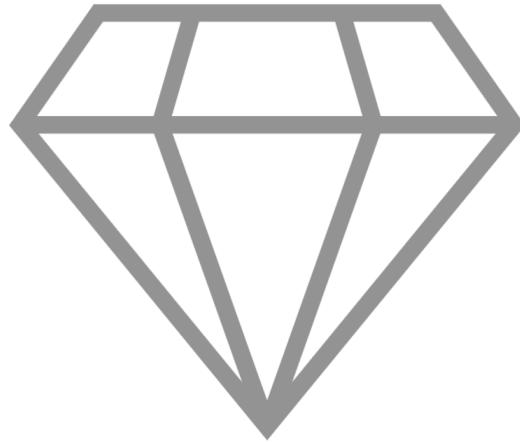


Versioning

Properties of a Reliable Application



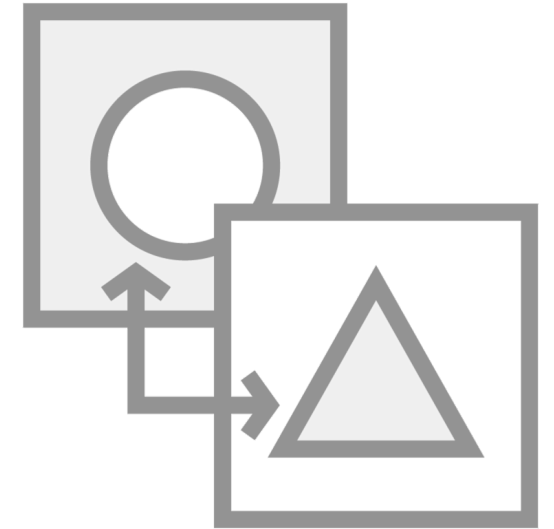
Idempotence



Immutability



Location
Independence

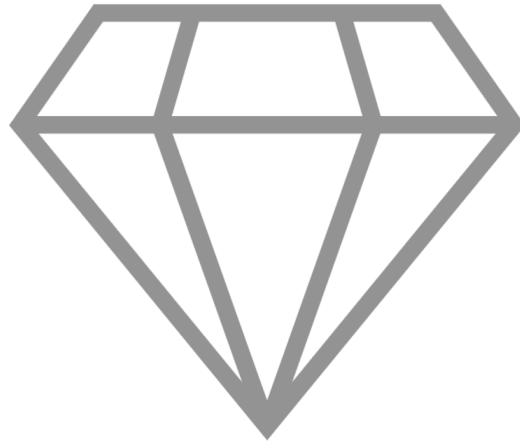


Versioning

Properties of a Reliable Application



Idempotence



Immutability



Location
Independence

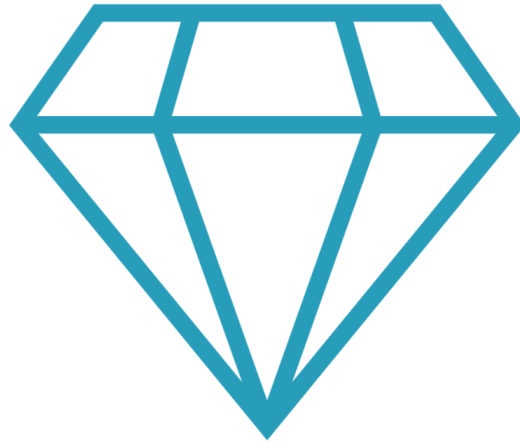


Versioning

Properties of a Reliable Application



Idempotence



Immutability



Location
Independence



Versioning



Sarah

Seasoned application developer and architect

Globomantics

GloboTicket

Defines IT strategy for GloboTicket



Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

Fallacies of Distributed Computing



Peter Deutch



Bill Joy



Tom Lyon



James Gosling



Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

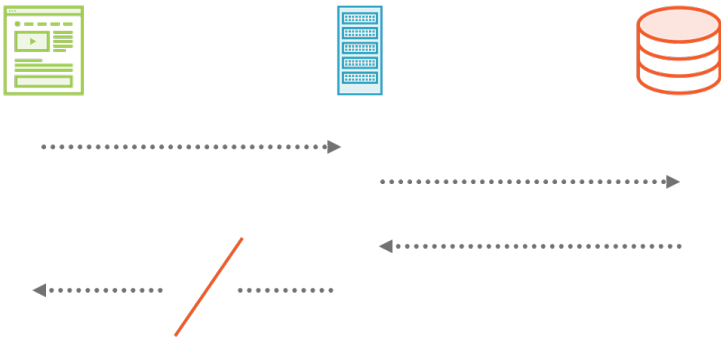
Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

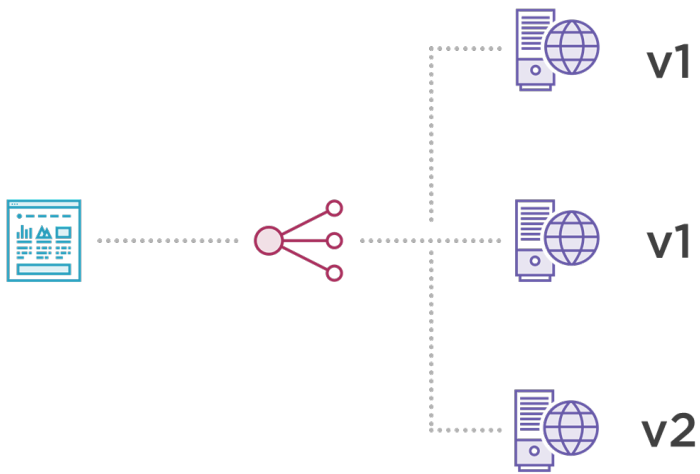
Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous



Fallacies of Distributed Computing



The network is reliable

Latency is zero

Bandwidth is infinite

The network is secure

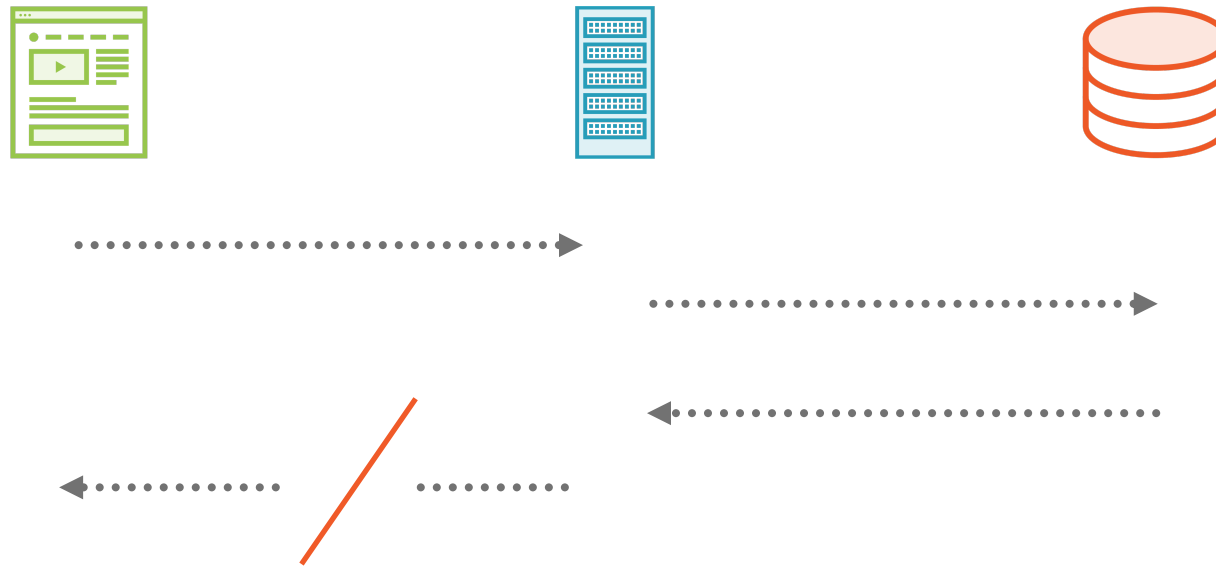
Topology doesn't change

There is one administrator

Transport cost is zero

The network is homogeneous

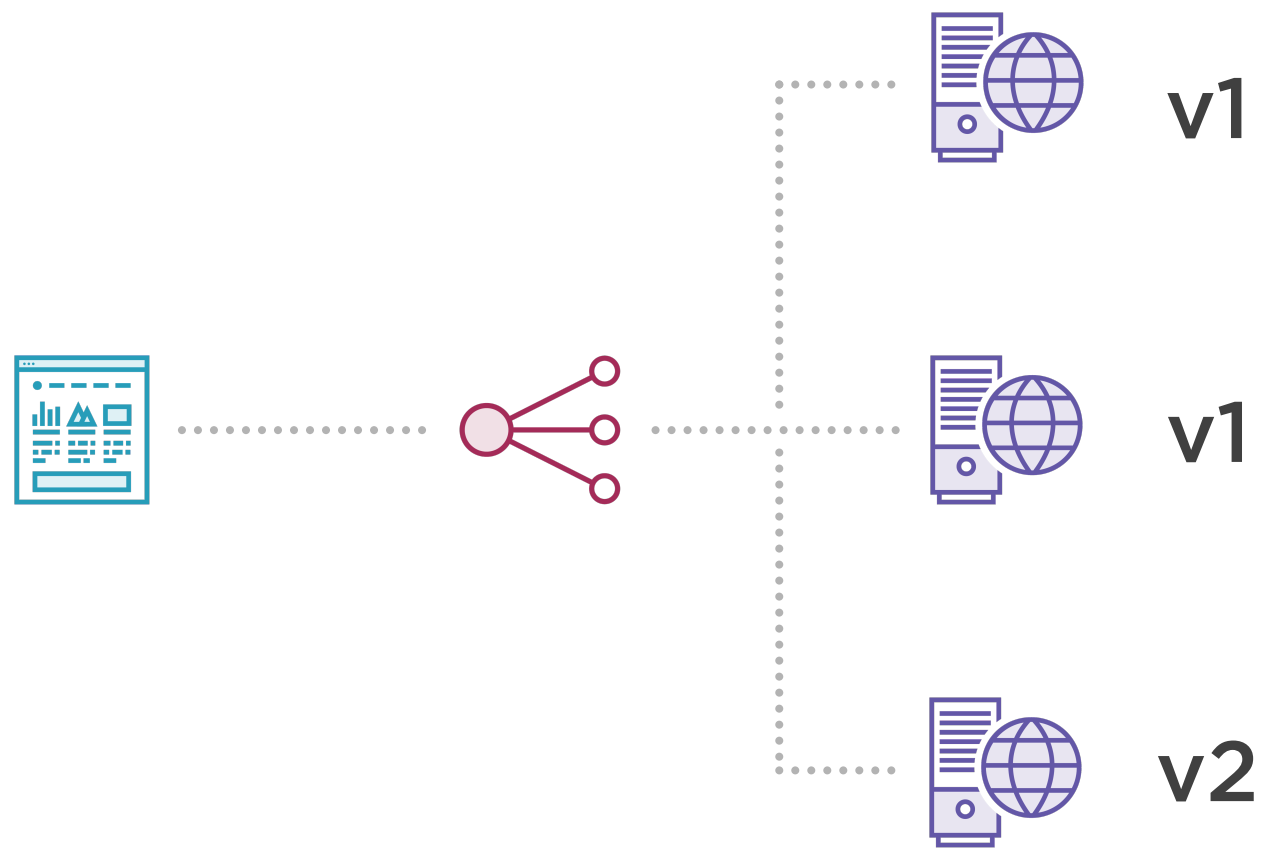
The Network is Reliable



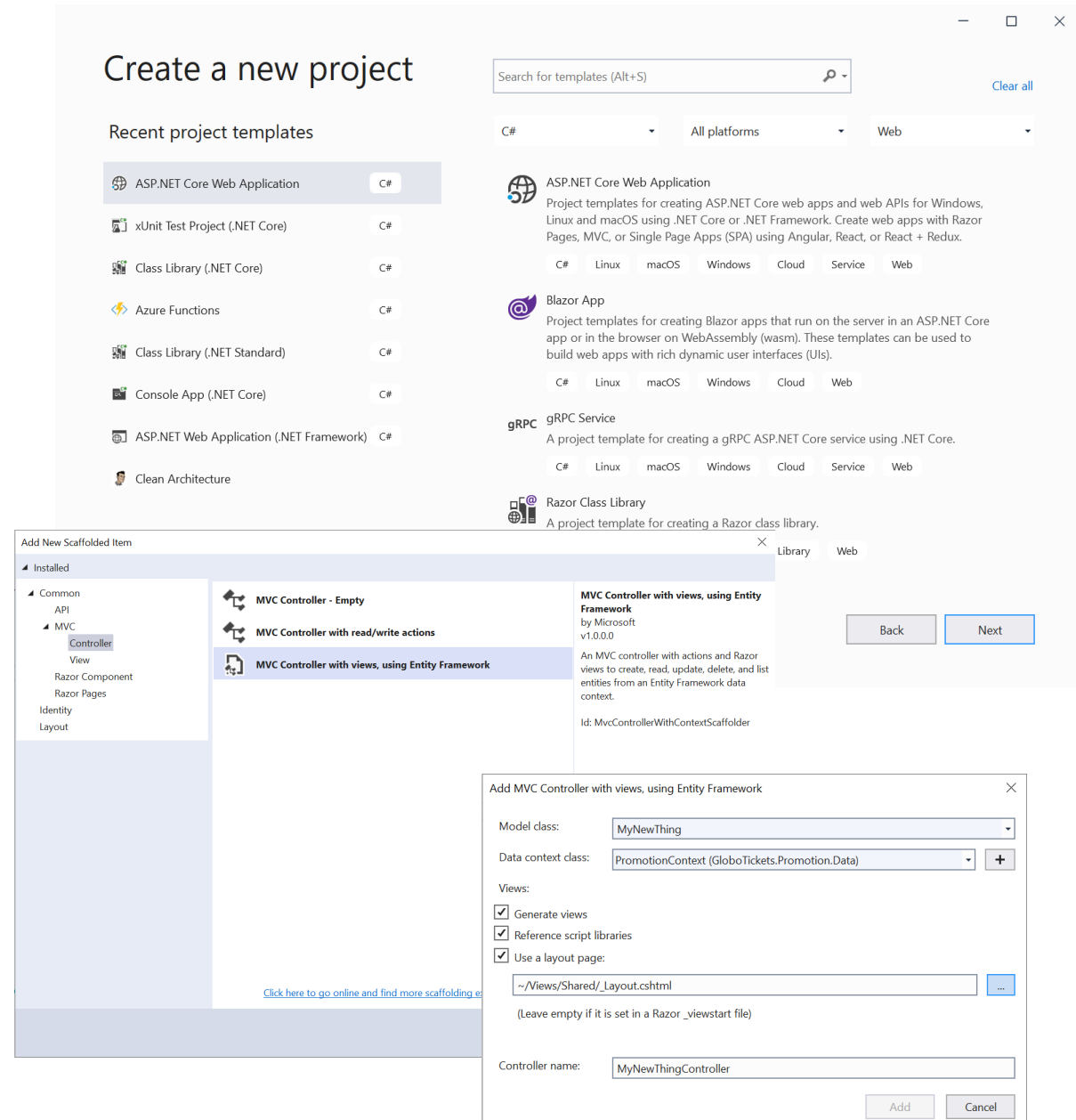
Topology Doesn't Change



The Network is Homogenous



Application template
Default database
design
Default API
CRUD operations



Default CRUD Mappings

Operations

Create

Read

Update

Delete

Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

POST

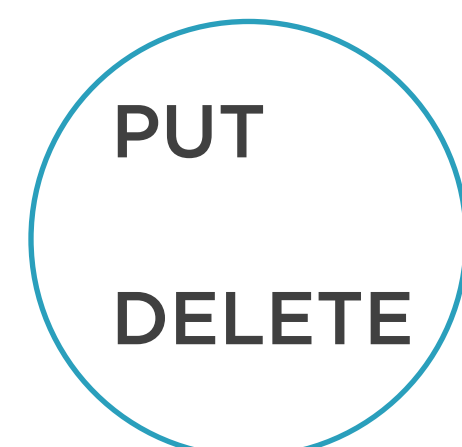
Not idempotent by default

GET

PUT

DELETE

Idempotent



Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

POST

Not idempotent by default

GET

PUT

DELETE

Idempotent

Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

POST

GET

PUT

DELETE

Idempotent

POST

Not idempotent by default

Idempotent

Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

Idempotent
POST

POST

GET

PUT

DELETE

Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

Idempotent
POST

POST

GET

PUT

DELETE

SQL

INSERT

SELECT

UPDATE

DELETE

Destructive
Immutable
Database

Default CRUD Mappings

Operations

Create

Read

Update

Delete

REST

Idempotent
POST

POST

GET

PUT

DELETE

SQL

INSERT

SELECT

UPDATE

DELETE

Immutable
Database



CONCERTS

Alexander Lemtov Live

Alexander Lemtov

1/21/2020

Join Alexander for his global tour. Alexander really needs no introduction since he has already mesmerized the world with his electronic ambient sound.

\$65 PER TICKET

QUANTITY:

2



PLACE ORDER

WARNING: Don't Click Twice!
Charges might be duplicated.



Typical REST API



POST /venues

INSERT INTO Venue

VenueID = 37

201 CREATED /venues/37

POST /venues

INSERT INTO Venue

VenueID = 38

201 CREATED /venues/38

Determine the Client's Intent

Create a second item

..... POST /venues ▶

..... POST /venues ▶

Retry a failed message

..... POST /venues ▶

..... POST /venues ▶



Client-side ID

Create a second item

POST /venues/abc-123
.....▶

POST /venues/xyz-789
.....▶

Retry a failed message

POST /venues/abc-123
.....▶

POST /venues/abc-123
.....▶



Client-side ID in Database



Client knows before creation

Can't use the auto-incrementing key

GUID is a good choice

Alternate key



Client-side ID API



POST /venues/abc-123

SELECT FROM Venue
WHERE Guid=abc-123

INSERT INTO Venue

201 CREATED /venues/abc-123

POST /venues/abc-123

SELECT FROM Venue
WHERE Guid=abc-123

201 CREATED /venues/abc-123

Idempotent POST
Alternate keys
Enforce idempotence
constraints



CQRS



Greg Young



Udi Dahan

**Command Query
Responsibility Segregation**

**Separate code paths for
commands and queries**



Command Query Responsibility Segregation



Commands

Persistent change
Return no information
Writes



Queries

No change
Return information
Reads

CQRS in Practice



Vladimir Khorikov

@vkhorikov

www.enterprisecraftsmanship.com



Separation of Concerns



Controllers



Data Access

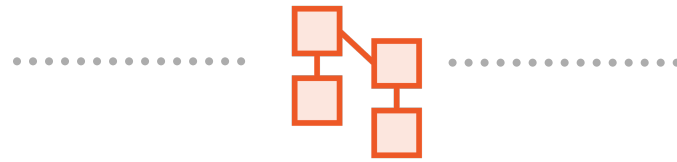
**Enforce
Idempotence**



Default Entity Framework Template



Controllers



Entity Objects

Primary key



Data Access



Separated Design



Controllers



Info Objects

Alternate key



Data Access



Venue Design



Venue
Controller



Venue Info
Venue GUID



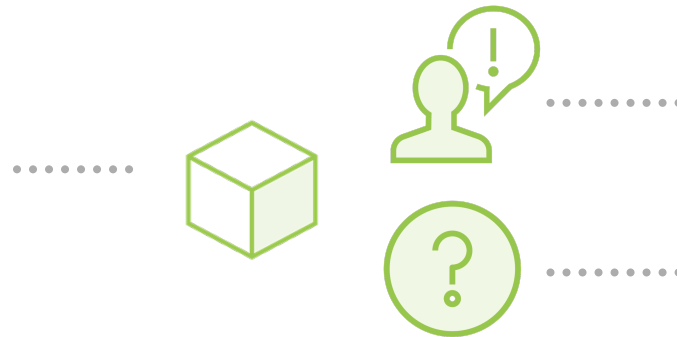
Venue
Repository



Venue Design



Venue
Controller



Venue Info
Venue GUID



Venue
Repository



```
Task SaveVenue(VenueInfo venueInfo);
```

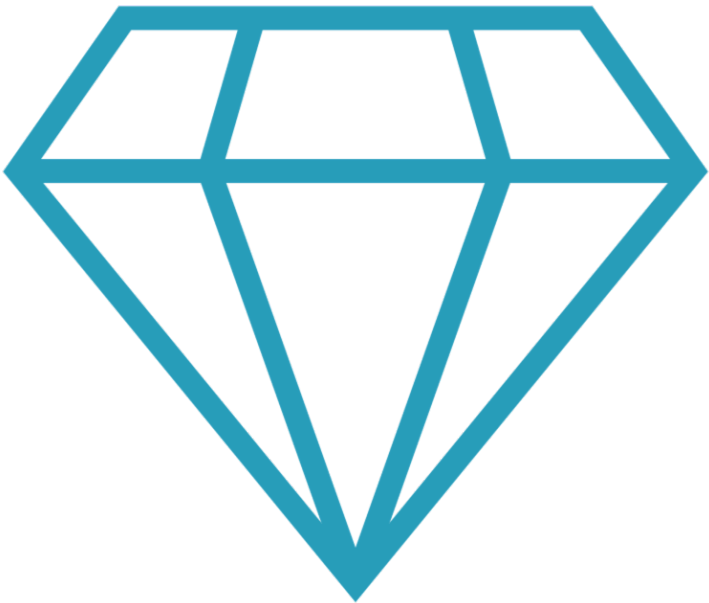
```
Task DeleteVenue(Guid venueGuid);
```

Venue Commands

Commands return no data



Immutable by Default



Make all data immutable

Reliable audit log

Do not destroy data

Preserve metadata

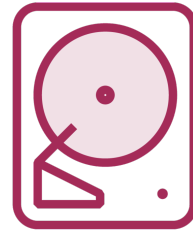
Promote eventual consistency



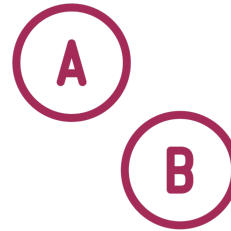
Immutability Changes Everything (2015)



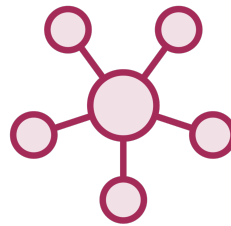
Pat Helland



Copy on Write



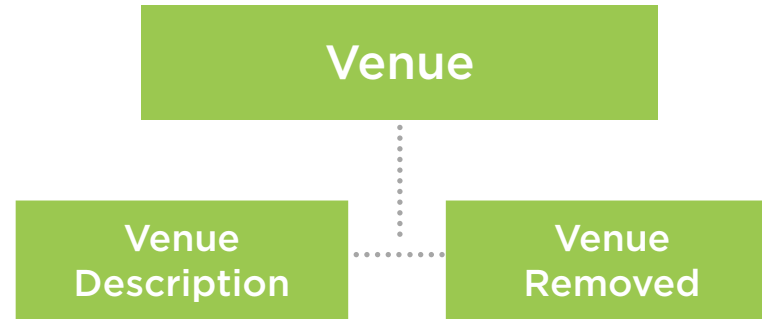
Snapshot Isolation



Map Reduce



Immutability Patterns



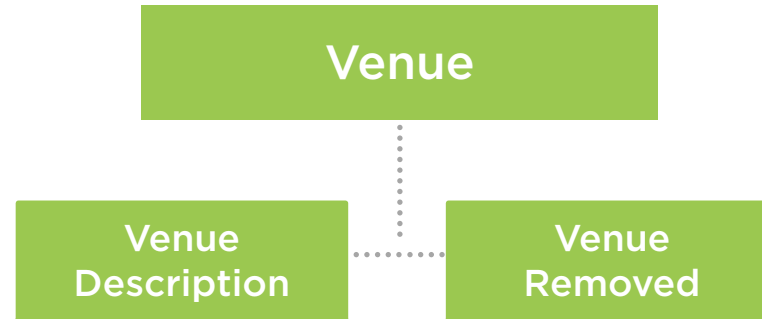
Snapshots



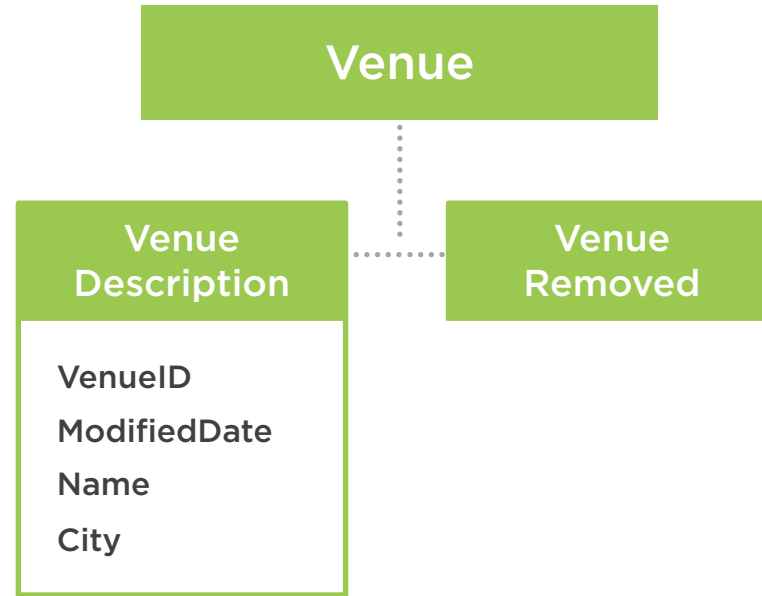
Tombstones



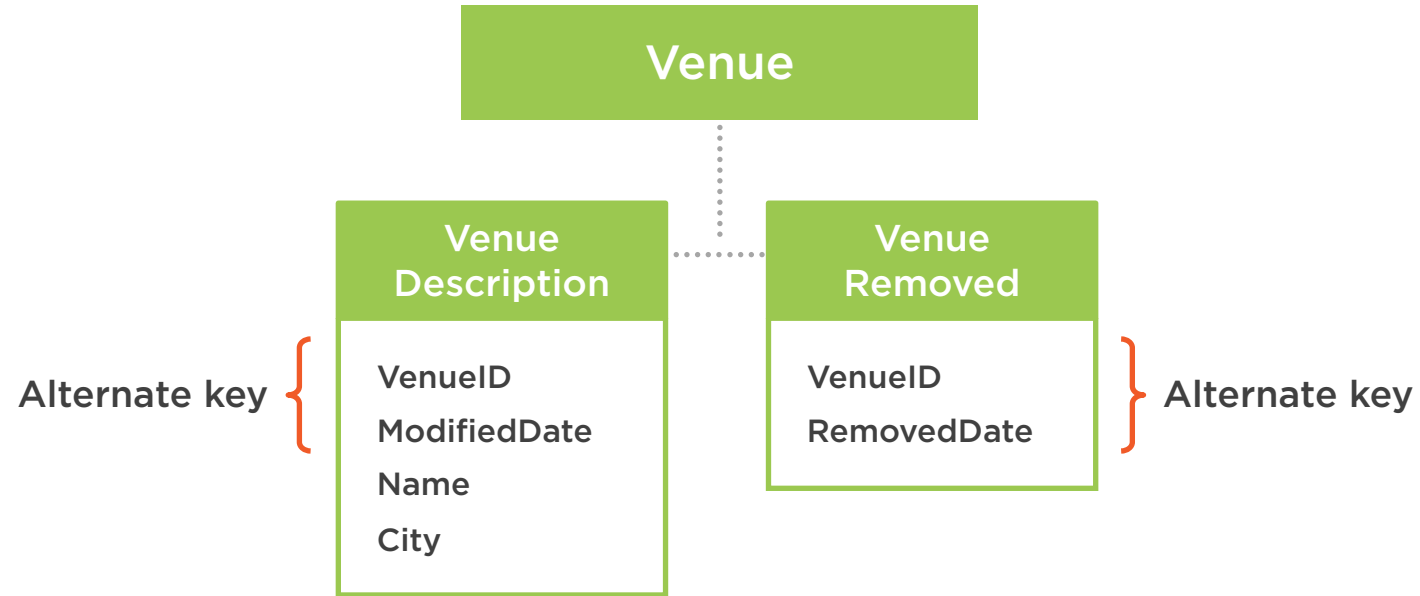
Immutability Patterns



Immutability Patterns



Immutability Patterns



Location Independence



Behavior does not depend upon location
Each instance produces the same behavior
Doesn't come for free



Auto-incrementing IDs



INSERT x

id: 437



INSERT x

id: 415



INSERT x

id: 319

Auto-incrementing IDs



INSERT x

id: 437



INSERT x

id: 415

INSERT y

id: 437



INSERT x

id: 319

INSERT z

id: 437

Location Independent Identifiers



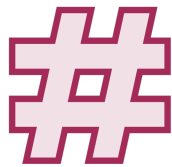
Alternate Key



Natural Key



Public Key



Hash



Content-addressed Storage



Content-addressed Storage



INSERT x

id: 0gQwXP



INSERT x

id: 0gQwXP



INSERT x

id: 0gQwXP

Content-addressed Storage



INSERT x

id: 0gQwXP



INSERT x

id: 0gQwXP

INSERT y

id: FRjIA7t



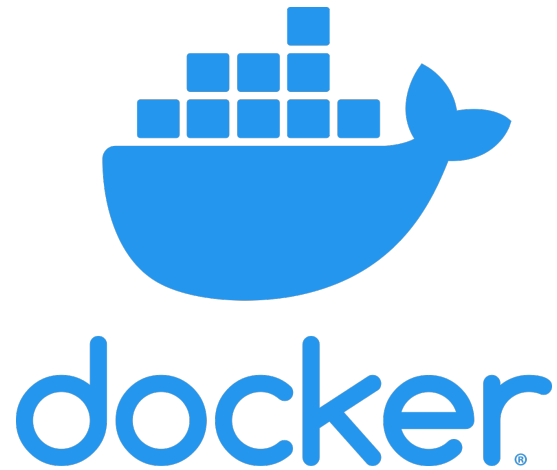
INSERT x

id: 0gQwXP

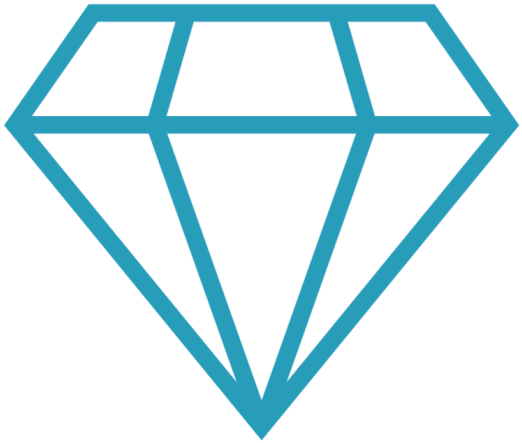
INSERT z

id: 4KwmuM

Implementations of Content-address Storage



Advantages of Content-address Storage



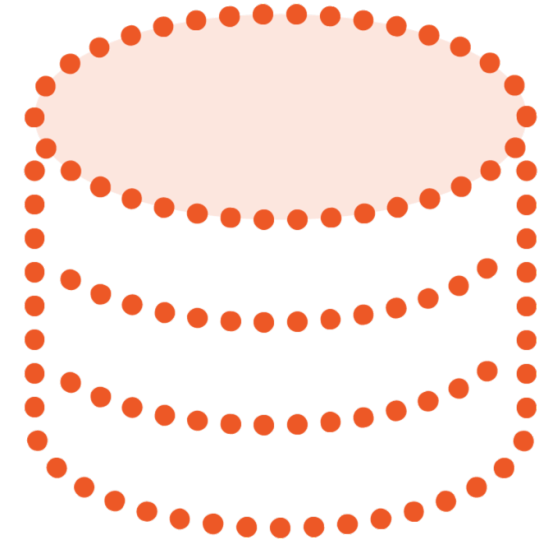
Naturally
Immutable



Naturally
Verifiable

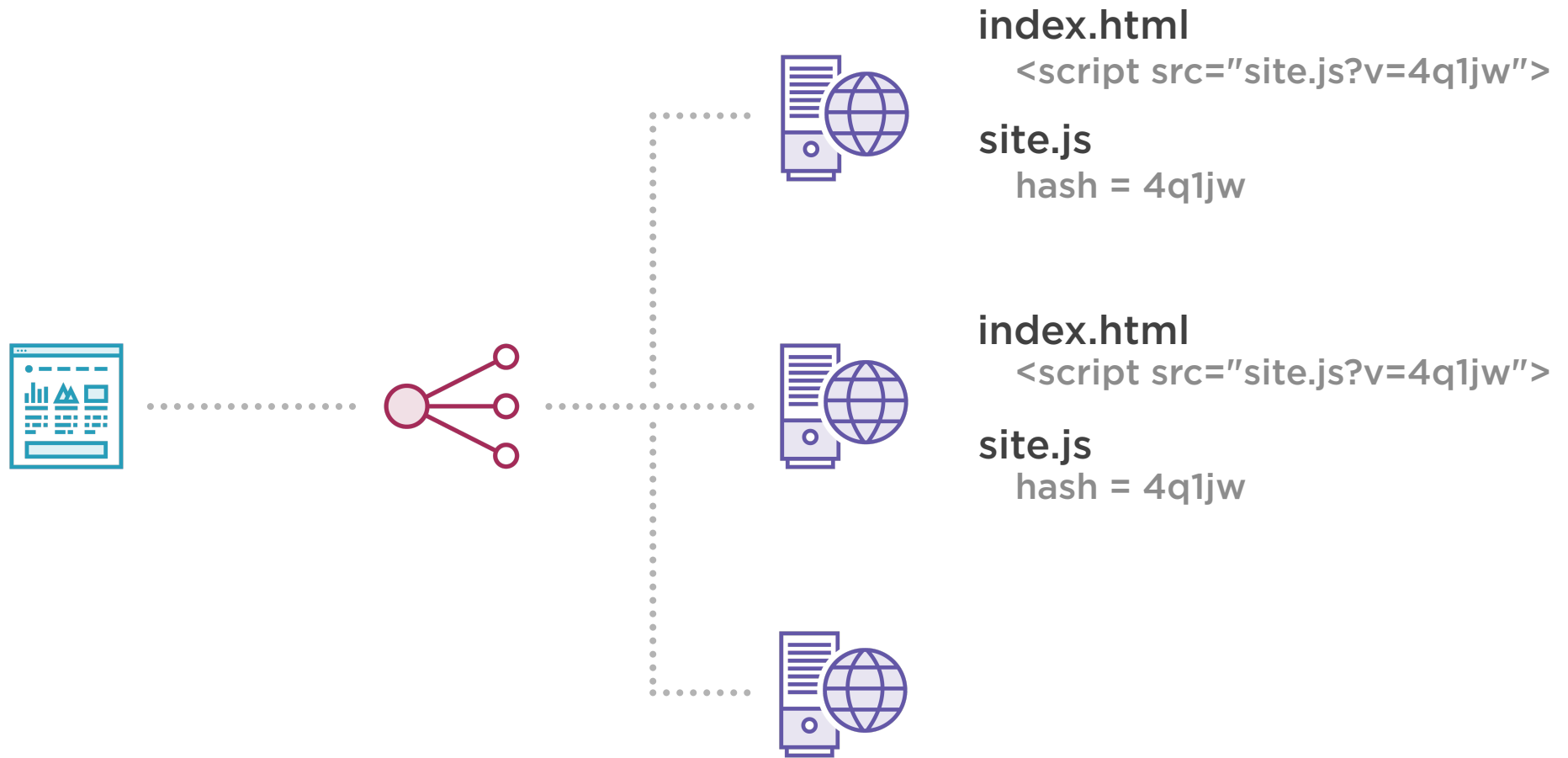


Naturally
Idempotent

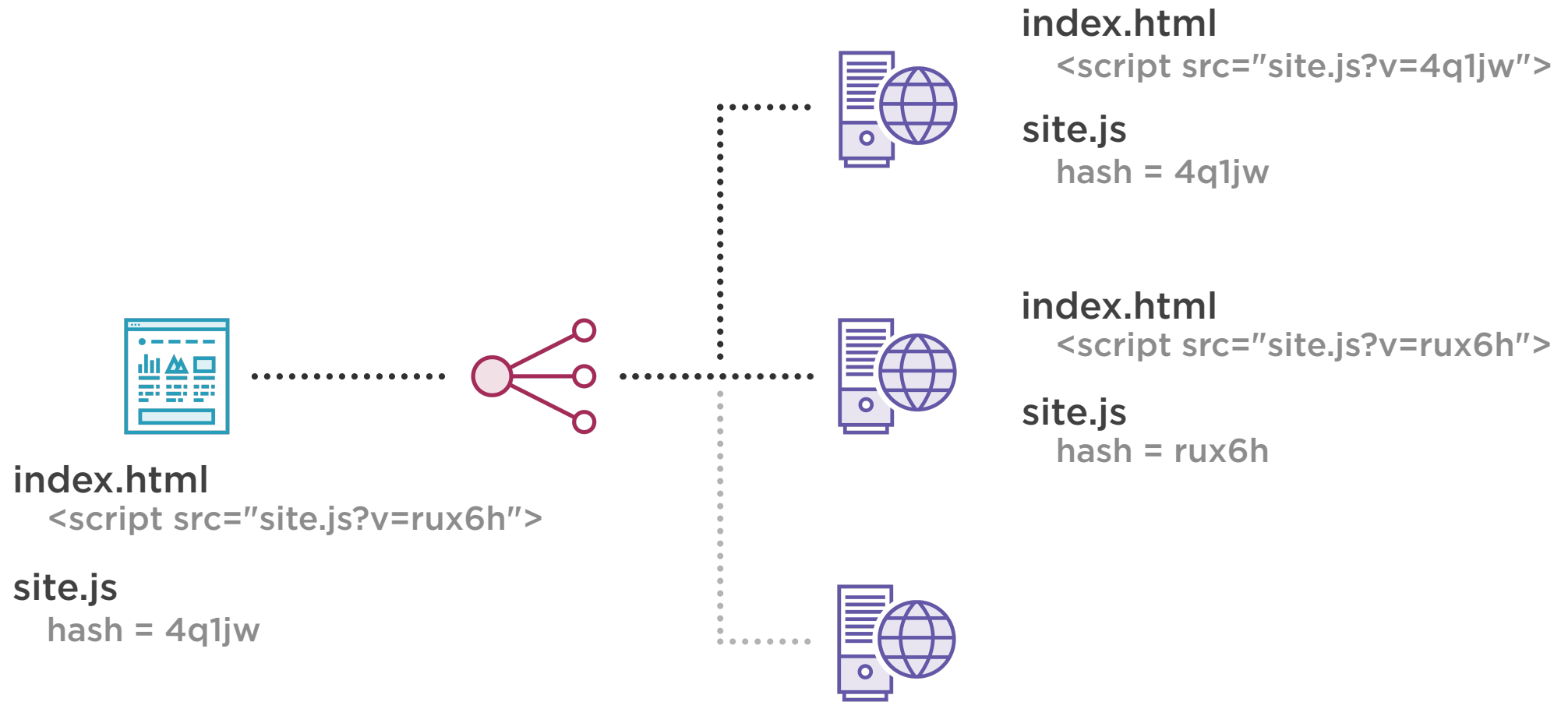


Solves Cache
Invalidation

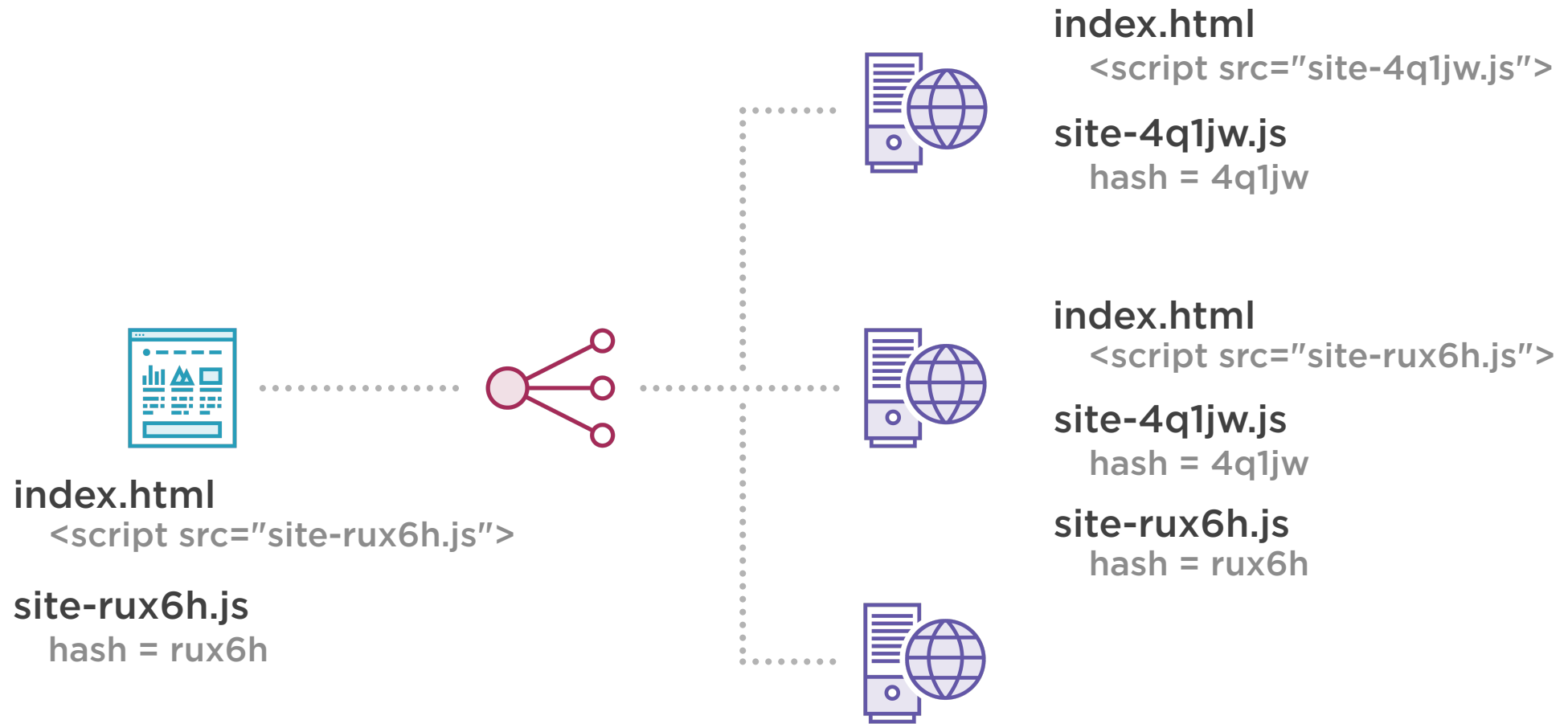
Problem with Cache Busting



Problem with Cache Busting



Problem with Cache Busting



Versioning



The network is not homogenous

Different versions

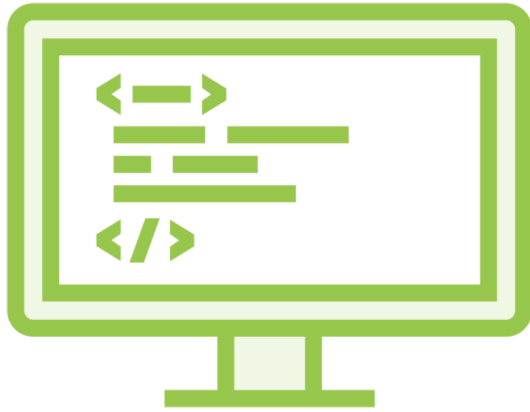
Different locations

Different times

Maintain contract



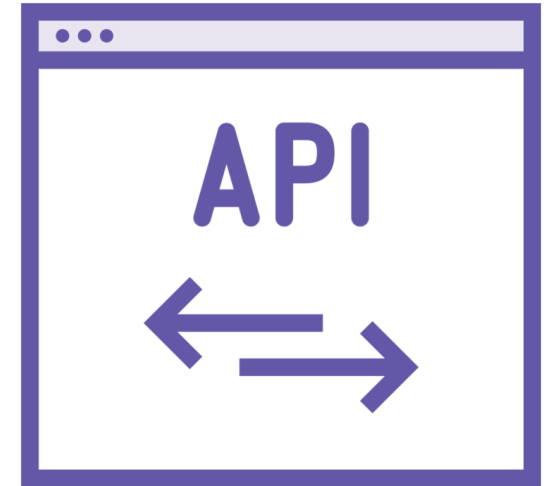
Components to be Versioned



Code

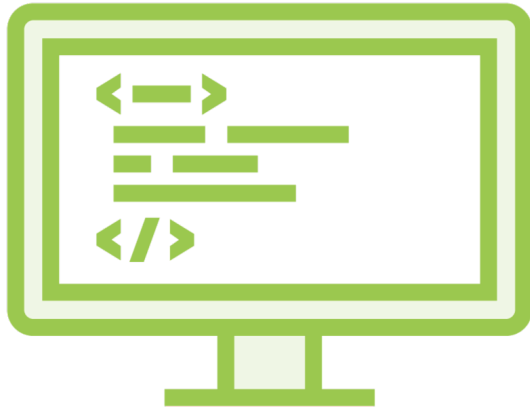


Data



API

Components to be Versioned

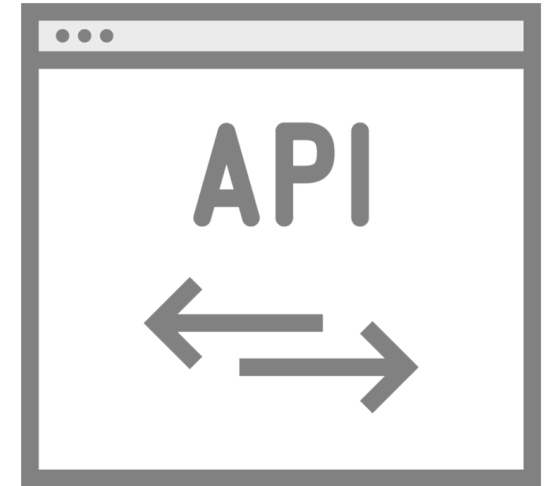


Code

Source control
CI/CD
Content-addressed storage

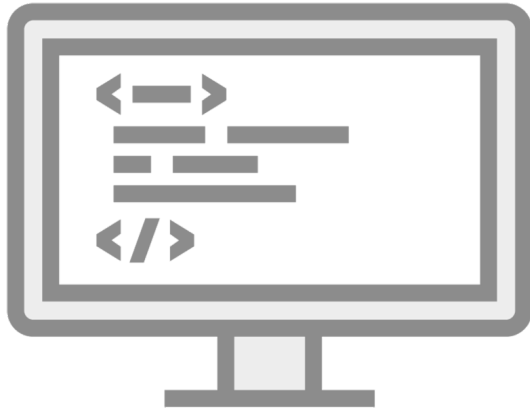


Data



API

Components to be Versioned

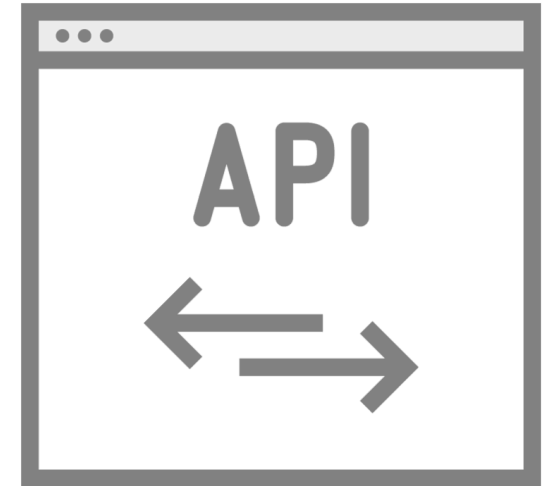


Code



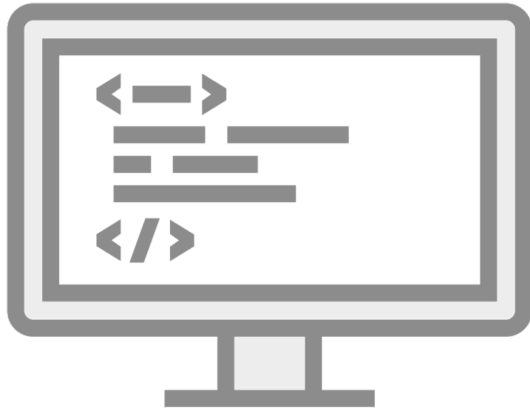
Data

Migrate to evolve
Immutable records
Favor new tables



API

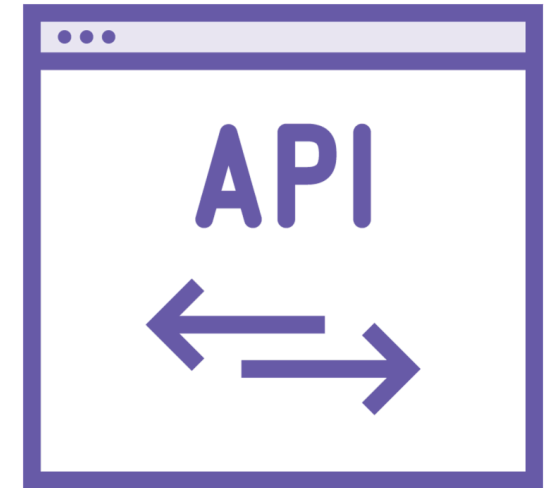
Components to be Versioned



Code



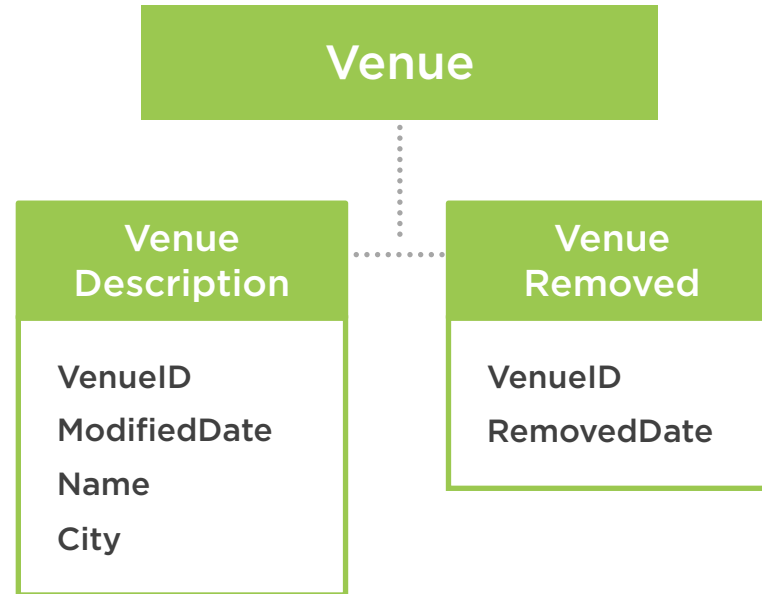
Data



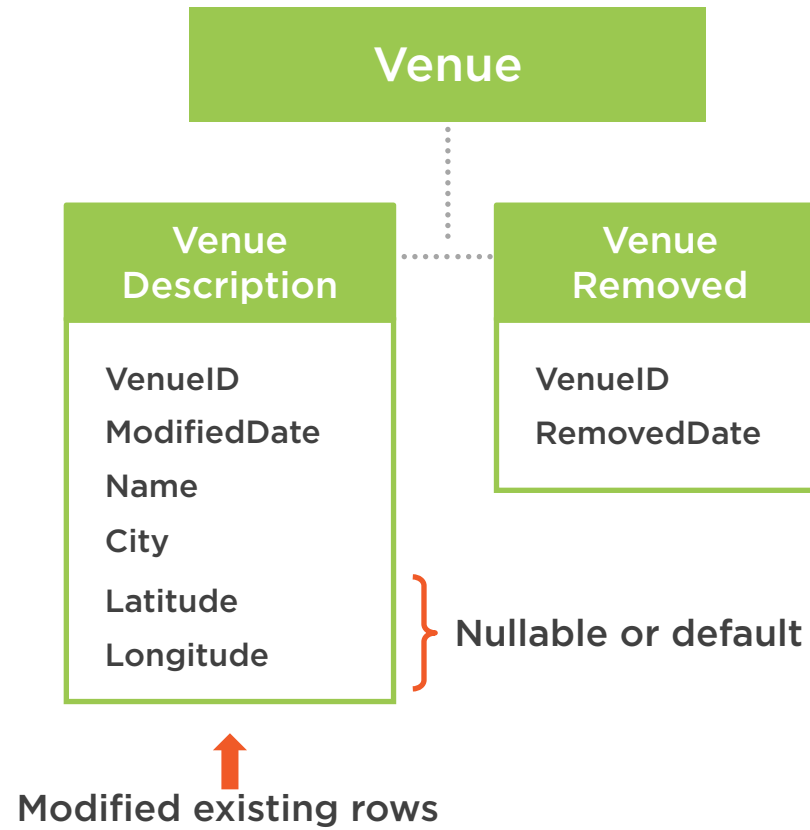
API

Application Lifecycle
Management
Additive structure

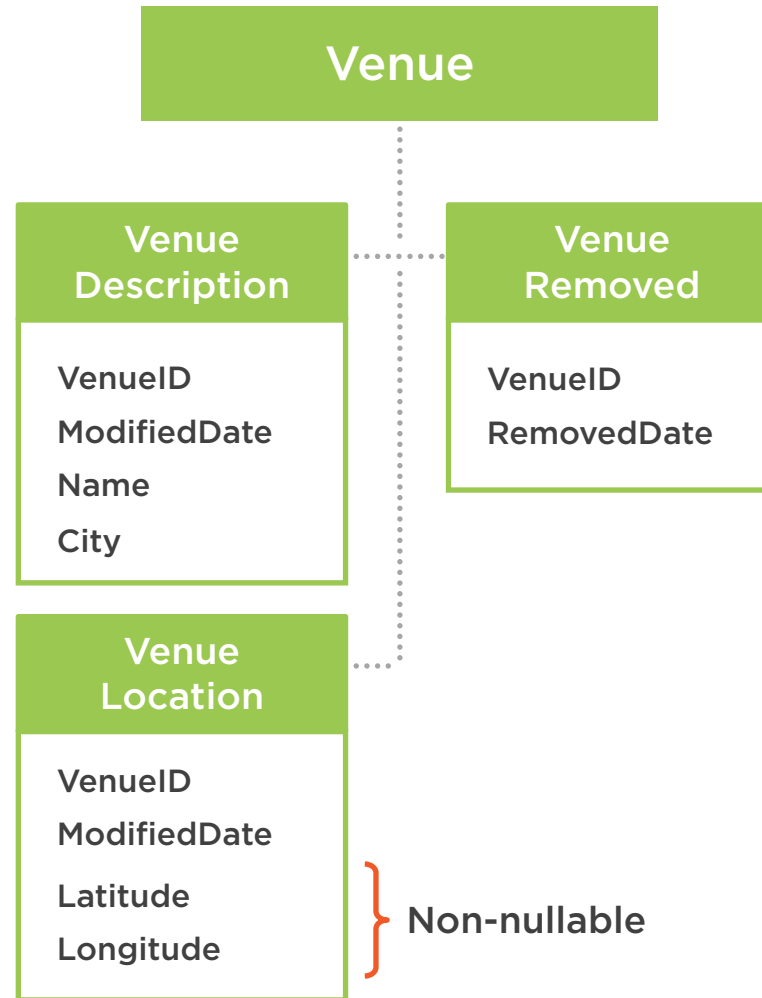
Venue Data



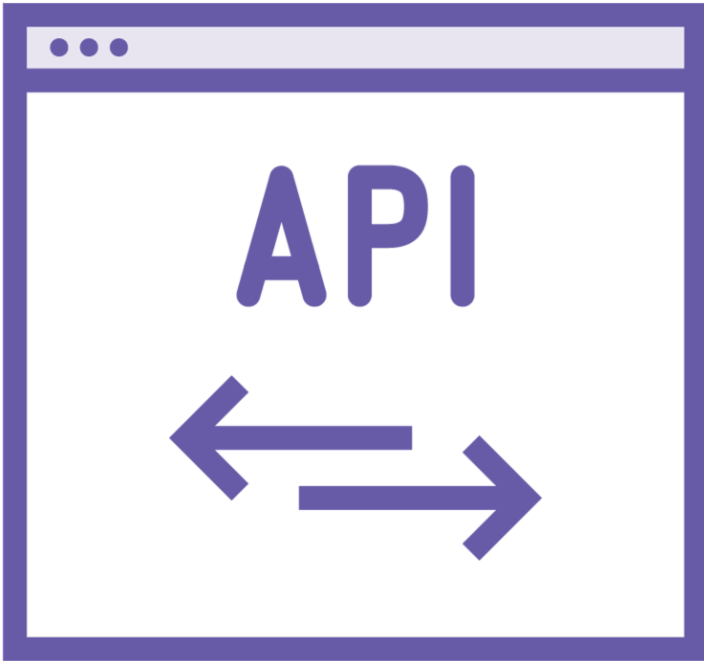
Venue Data



Venue Data



API Contract



Endpoints

Messages

Request

Response

Server-specified

Two-sided

Incremental



Additive Structure

```
{  
  "venueGuid": "E12AEEB5-9443-4AD4-B394-EFB37B0DD67F",  
  "description": {  
    "name": "Madison Square Garden",  
    "city": "New York, NY",  
    "modifiedDate": "2020-11-22T22:09:27.993Z"  
  }  
}
```



Additive Structure

```
{  
  "venueGuid": "E12AEEB5-9443-4AD4-B394-EFB37B0DD67F",  
  "description": {  
    "name": "Madison Square Garden",  
    "city": "New York, NY",  
    "modifiedDate": "2020-11-22T22:09:27.993Z"  
  },  
  "location": {  
    "latitude": 40.7505,  
    "longitude": -73.9934,  
    "modifiedDate": "2020-11-28T05:06:39.215Z"  
  }  
}
```



Other Versioning Strategies



URL

- /api/v1/venues
- /api/v2/venues

Content Types

- Accept: application/venue.v1+json
- Content-Type: application/venue.v2+json

Query String

- /api/venues?v=v1
- /api/venues?v=v2

Avoid it!

- GraphQL

Versioning and Evolving Microservices in ASP.NET



Mark Heath

@mark_heath www.markheath.com

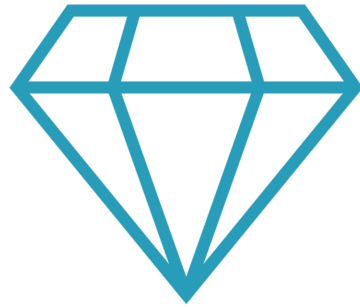


Overview of a Reliable Application



Idempotence

Alternate keys
Client-side IDs



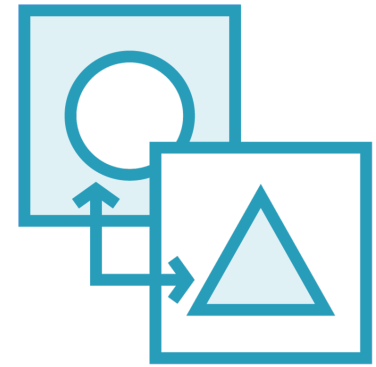
Immutability

History of intent
Metadata
Insert-only database
Snapshots and tombstones



Location Independence

Hide auto-incremented ID
Content-addressed storage



Versioning

New tables
Additive structure

Up Next: Connecting Services

